Build your own

AI Agent

Microsoft Agent Framework & AG-UI

Medhat Elmasry & Jasmin Goh

AG-UI: The Agent-User Interaction Protocol

AG-UI is an event-based protocol, created in May 2025, for **connecting AI agents to user-facing applications**.

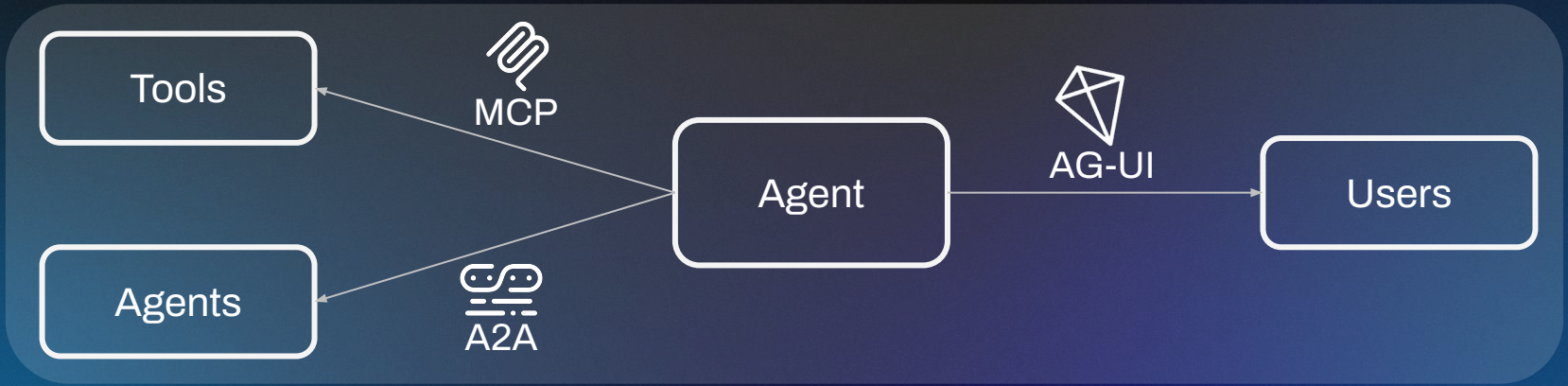
It acts as a **bi-directional bridge** between:

- Your agentic backend
- Your user-facing frontend

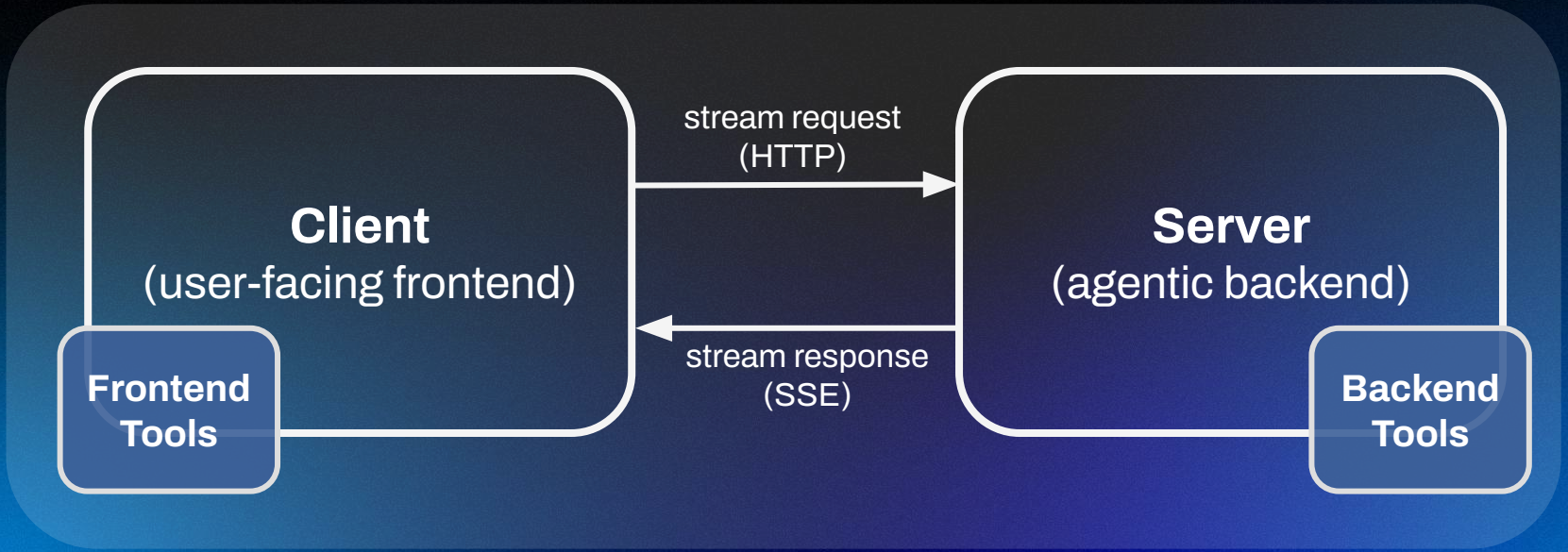
With AG-UI, you can **avoid complex ad-hoc wiring** for agentic features.

The Agent Protocol Stack

- **MCP** standardizes agent-tool integration
- **A2A** enables agent-to-agent communication
- **AG-UI** bridges agents and users



AG-UI Architecture



Why AG-UI?

1. **Remote Agent Hosting**
   - Agents can be deployed as services **accessible by multiple clients** (web, mobile)
2. **Real-time Streaming**
   - Agent **responses can be streamed** using Server-Sent Events (SSE)
3. **Backend and Frontend Tools**
   - **Results** from backend tools are **automatically streamed to client**
   - **UI components can be modified** via frontend tools
4. **Human-in-the-Loop**
   - **User confirmation can be requested** before tool execution

AG-UI Blazor demo

Workshop

You'll be building a **console app** with a client-server architecture

1. **Server setup**
   Build an AG-UI server to host your AI agent
2. **Client setup**
   Create a UI that streams responses from the agent
3. **Tools**
   Add tools the agent can call
4. **Human-in-the-Loop**
   Add tools that require user approval to proceed

Workshop: 1. Server Setup

Build an AG-UI server to host your AI agent

By the end of this, you'll have:

- A running AG-UI server
- An AI agent hosted on the server
- An HTTP endpoint listening for requests

Workshop: 2. Client Setup

Create a UI that streams responses from the agent

By the end of this, you'll have:

- A client connected to the AG-UI server
- User input sent to the agent
- Agent responses streamed and displayed in the UI

Workshop: 3. Tools

Add tools your agent can call to perform tasks

By the end of this, you'll have:

- A backend tool that runs on the server
- A frontend tool that runs on the client
- An agent that decides when to call each tool

Workshop: 4. Human-in-the-Loop

Add tools that require user approval to proceed

By the end of this, you'll have:

- A tool that requests user approval
- An agent that pauses while waiting for input
- The ability to approve or reject action and resume execution

Time to build!

(Connect to internet via your own hotspot, install .NET SDK, VS Code w/ C# Dev Kit extension)

Head on over to: https://bit.ly/netbc-agui

0. **GitHub Token**: Get your GitHub PAT (if you don't have it yet)
1. **Server setup**: Build an server to host your agent
2. **Client setup**: Create a UI to streams agent response
3. **Tools**: Add tools the agent can call
4. **Human-in-the-Loop**: Add tools that require user approval

Stuck?

Call Medhat or Jasmin!

We'll be going around the room making sure you're good

Done…?

Build you own tools!

Connect to an API, analyze uploaded CSV, approve recipe ideas…

Resources

- Our GitHub repo: https://bit.ly/netbc-agui
- AG-UI:
  - https://docs.ag-ui.com
- AI Agent protocols:
  - https://www.ibm.com/think/topics/ai-agent-protocols
  - https://modelcontextprotocol.io
  - https://a2a-protocol.org
- AG-UI Integration with Microsoft Agent Framework: https://learn.microsoft.com/en-us/agent-framework/integrations/ag-ui
- Building an AG-UI project: https://youtu.be/tDQc6lZUbYc?si=Uc4TE2Sn_BREIPtp

Thank you

Take what you learned, build something yours, and come tell us about it at our next event!

Questions?

Medhat Elmasry
medhat@elmasry.ca

Jasmin Goh
hello.jasmin.noe@gmail.com